CODE :-

```
#include <stdio.h>
#include <ctype.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int value)
{
   if (top == MAX - 1)
   {
      printf("Stack Overflow\n");
      return;
   }

   top++;
   stack[top] = value;
}

int pop()
{
   if (top == -1)
   {
      printf("Stack Underflow\n");
      return 0;
   }

   return stack[top--];
```

```
}

int evaluatePostfix(char expression[])
{
   int i;
   int operand1, operand2;
   int result;
   char ch;

   for (i = 0; expression[i] != '\0'; i++)
   {
      ch = expression[i];

      if (isdigit(ch))
      {
         push(ch - '0');
      }
      else if (ch == '+' || ch == '-' || ch == '*' ||
            ch == '/' || ch == '%')
      {
         operand2 = pop();
         operand1 = pop();

         switch (ch)
         {
            case '+':
               result = operand1 + operand2;
               break;

            case '-':
               result = operand1 - operand2;
```

```
                break;

            case '*':
                result = operand1 * operand2;
                break;

            case '/':
                result = operand1 / operand2;
                break;

            case '%':
                result = operand1 % operand2;
                break;
        }

        push(result);
      }
   }

   return pop();
}

int main()
{
   char expression[MAX];
   int result;

   printf(" POSTFIX EXPRESSION EVALUATION \n");

   printf("Enter postfix expression: ");
   scanf("%s", expression);
```

```
    result = evaluatePostfix(expression);

    printf("Result = %d\n", result);

    return 0;
}
```

Output :-

```
POSTFIX EXPRESSION EVALUATION
Enter postfix expression: 35+4*

Result = 32

POSTFIX EXPRESSION EVALUATION
Enter postfix expression: 23+5*

Result = 25
```

WRITEUP :-

Assignment no :- 8

Problem Statement :-
Write a C program to evaluate postfix expression using stack.

Objective :-
1) To study stack and its operations
2) To study the importance of expression evaluation

Theory :-
Expression evaluation is the process of calculating the value of an arithmetic expression. A stack is the most suitable data structure because it follows the Last In First Out (LIFO) principle, which matches the order in which operators and operands are processed.
Expression evaluation is commonly performed in
- Postfix (Reverse Polish Notation)
- Prefix (Polish Notation)

Infix expressions are usually converted to postfix or prefix before evaluation.

Eg :-
Evaluate the expression: 3 5 + 4 *

Symbol	Operation	Stack
3	Push 3	3
5	Push 5	3,5
+	Pop 5, pop 3 perform 3+5 Push 8	8
4	Push 4	4,8
*	Pop 4, Pop 8 perform 4*8 Push 32	32

Implementation :-
- Platform :
-> 64-bit Open source Linux or its derivative
-> Open Source C Programming tool like gcc/g++ Editor

Test Cond" :-
- Input postfix expression
-> Evaluate the value of expression.

Pseudo Code :-
- POSTFIX_EVALUATION (expression).
1) Initialize an empty stack.
2. Scan the postfix expression from left to right.
3. If the scanned symbol is an operand :
   PUSH the operand into the stack.
4. If the scanned symbol is an operator :-
   a. POP the top element & store it as operand 2.
   b. POP the next element & store it as operand 1.
   c. Perform :
      operand 1 operator operand 2
   d. Push the result into the stack.
5. Repeat steps 3 & 4 until the complete expression is scanned.
6. POP the final element from the stack.
7. Display the final value.
8. Stop.

5. After scanning the complete expression:
   POP the final expression.
6. Display the infix expression.
7. Stop

Time Complexity :-

Operation	Time Complexity
Push	O(1)
Pop	O(1)
IsEmpty	O(1)
IsFull	O(1)
Infix → Postfix	O(n)
Postfix → Infix	O(n)

Conclusion :-
Thus, implemented Stack Operations using array concepts. This system is able to apply different expression conversions.

FAQs :-

1) What is the advantage of prefix and postfix over infix expression?
-> Prefix and postfix expressions are easier for computers to evaluate because they do not require parentheses and operator-precedence rules during evaluation. They provide an unambiguous order of operations.

2) Why is stack used in expression conversion?
-> It is used to temporarily store operators and parentheses during expression conversion. It helps arrange the operators in the correct order according to their precedence.

3) Give stack full and stack empty cond" for stack.
-> Stack full : When top == MAX-1, the stack is full. A further push operation causes overflow.
- Stack Empty - When top == -1, the stack is empty. A pop operation causes stack underflow.

5/11/25